Theory of Computation

What can be done by computing machines?
Computability

What can realistically be done by computing machines?
Complexity

Where on earth to start?
Turing Machines

Edwin Blake, Tommie Meyer, Nathan Geffen & Sonia Berman

Turing Machines can do everything a computer can do.

There are problems Turing Machines cannot solve !



Edwin Blake, Tommie Meyer, Nathan Geffen & Sonia Berman

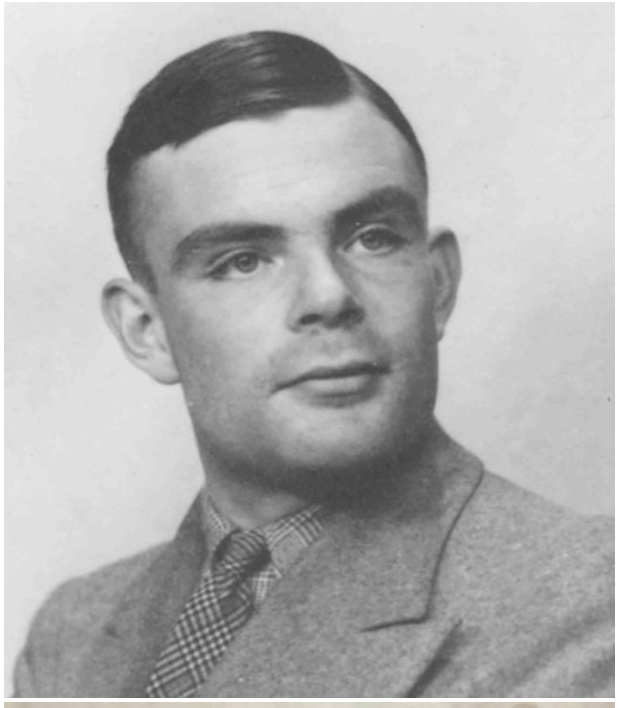
Turing Machines

An idealized type of machine designed by Alan Turing in 1936

with the aim of capturing the essential elements of algorithms

Minimum requirements for maximum algorithmic power,

like a human solving a problem with (unlimited) pen and paper





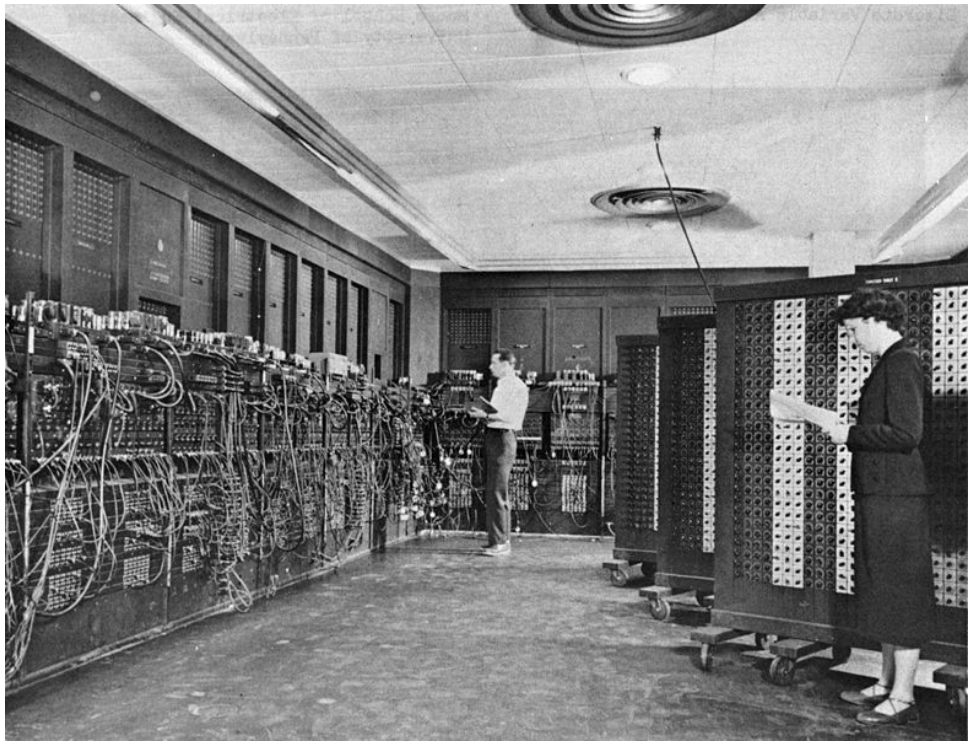
Why the Fuss?

No one has been able to come up with a more powerful machine than a Turing Machine

- Church-Turing Thesis

  Any algorithm or computation can be represented by a TM

  - Accepted by most.



ENIAC (1947-1955) Philadelphia, Pennsylvania

Glen Beck (background) and Betty Snyder (foreground) program the ENIAC

http://www.turing.org.uk/turing/

For interest only.

Not examinable.

Alan Turing, 1912–1954

- Developed both theory of computation and showed practical application of computers

1936: The Turing machine, computability, universal machine

1938-39: Introduced to German Enigma cipher machine

1939-40: The Bombe, machine for Enigma decryption

1939-42: Breaking of U-boat Enigma, saving battle of the Atlantic

1943-45: Chief Anglo-American crypto consultant. Electronic work.

1946: Computer and software design leading the world.

1947-48: Programming, neural nets, and artificial intelligence

1949: First serious mathematical use of a computer

1950: The Turing Test for machine intelligence

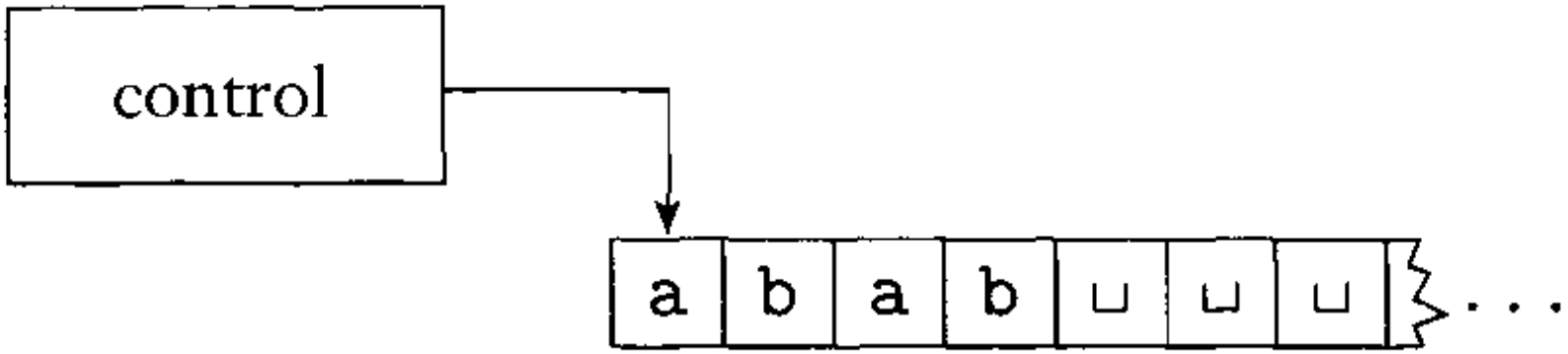
1951: Elected FRS. Non-linear theory of biological growth

1952: Arrested as a homosexual, loss of security clearance

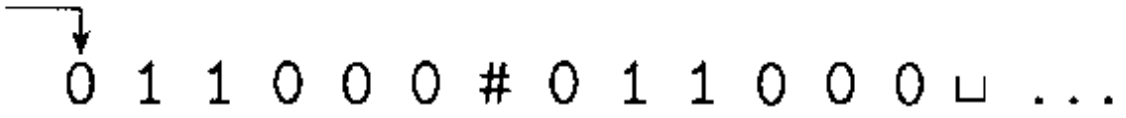
1953-54: Unfinished work in biology and physics

1954 (7 June): Death (suicide) by cyanide poisoning, Wilmslow, Cheshire.

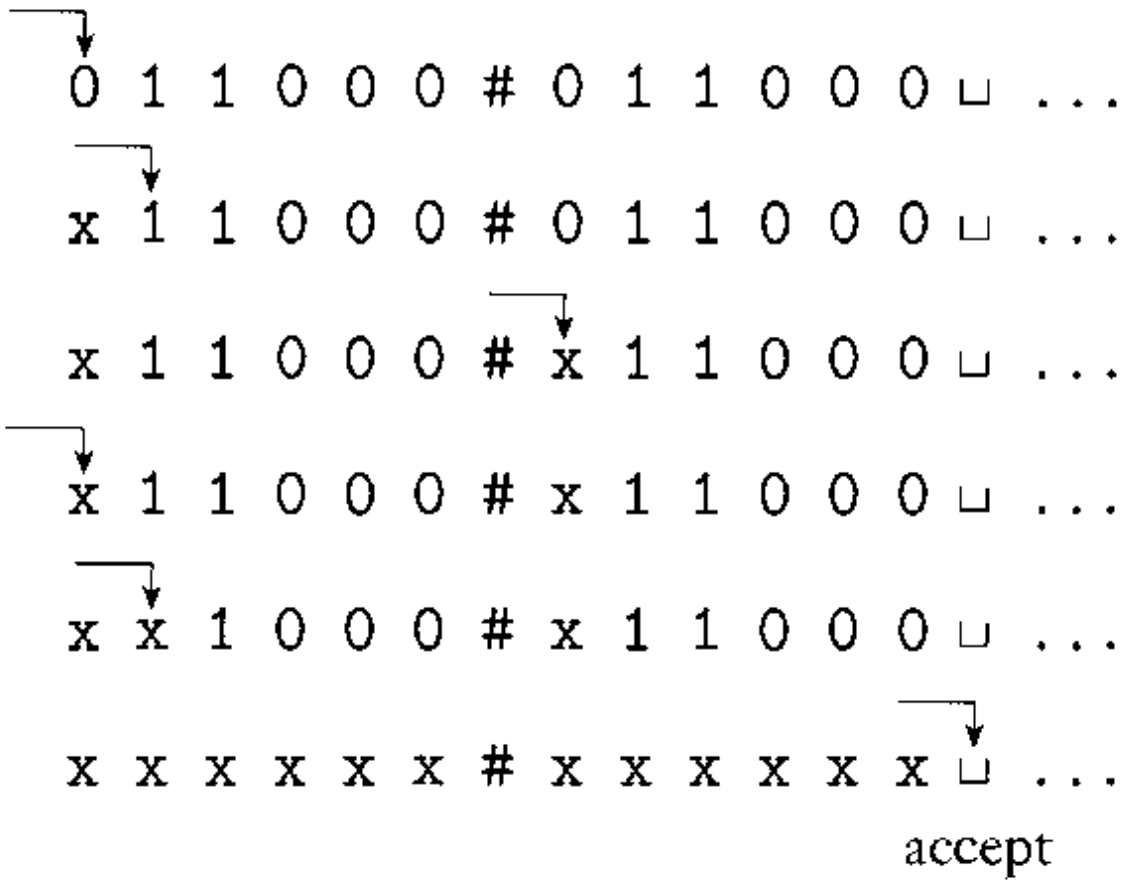
Schematic of a Turing Machine

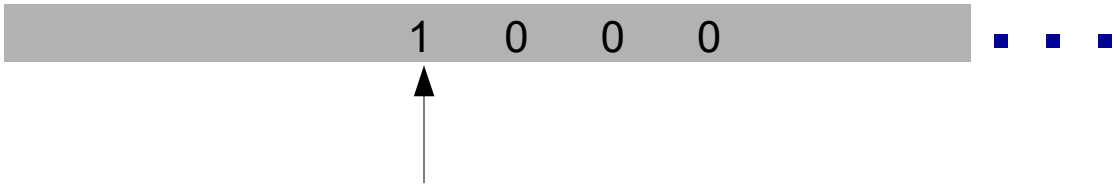


$$B = \{ w\#w \mid w \in \{0,1\}^* \}$$



$B = \{ w\#w \mid w \in \{0,1\}^* \}$





Initial State: q0

Halt state: qAccept

$\delta(q0,0) = (q1,R)$ // If in q0 and symbol is 0, change to q1 and go right

$\delta(q0,1) = (q1,R)$ // If in q0 and symbol is 1, change to q1 and go right

$\delta(q1,0) = (q0,R)$ // If in q1 and symbol is 0, change to q0 and go right

$\delta(q1,1) = (q0,R)$ // If in q1 and symbol is 1, change to q0 and go right

$\delta(q0, \Diamond) = (qAccept, 1)$ // if in q0 and symbol is space, halt and write 1

$\delta(q1, \Diamond) = (qAccept, 0)$ // if in q1 and symbol is space, halt and write 0

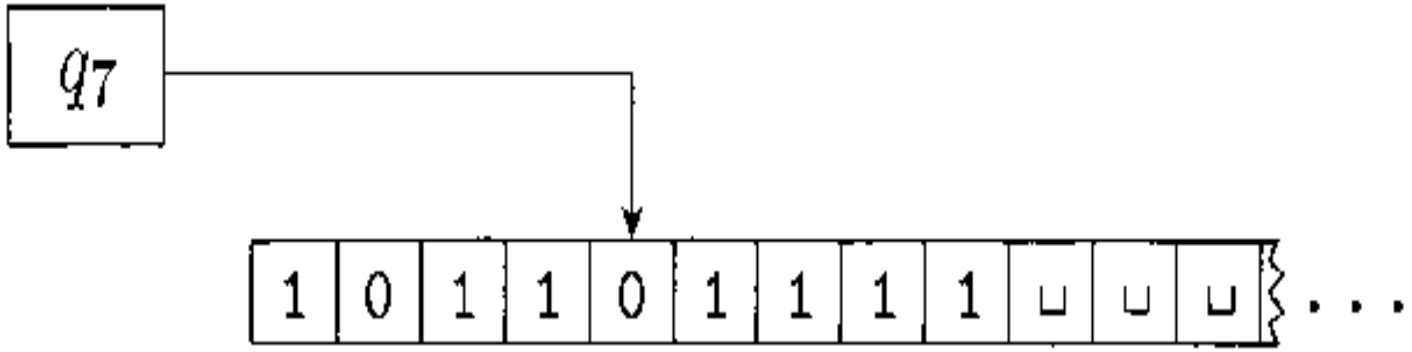


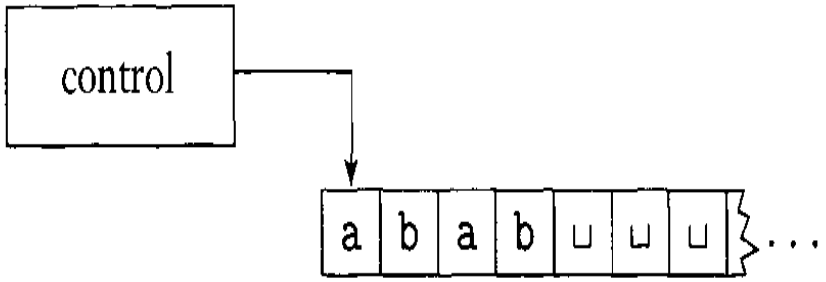
FIGURE 3.4
A Turing machine with configuration $1011q_701111$

Suppose we had $\delta(q_7, 0) = (q_2, R)$

If the current configuration is $1011q_701111$ what will the next configuration be?

$10110q_21111$

How will it all end ?

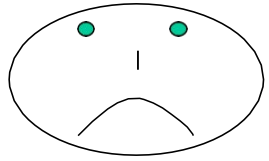


One of 3 things can happen to a Turing Machine:

1. Terminate in the **halt** state



2. **Crash** horribly by going off the left end, or reaching a situation where no action is defined



3. **Go on and on** and on and on and on and on and on and on and on and on and on and on and on and on ….

A Turing machine comprises:

- A **tape**, with equal sized blocks or cells, that extends indefinitely to the right, and a **tape head** positioned at one of these cells
- An **input alphabet** of input symbols that can appear on its tape
- A **tape alphabet** of symbols that can appear on its tape
- A **set of States**, S, **one of which is the initial state**, and some of which are **halt states** (machine stops in halt state)
- A **state table** consisting of entries : current state, current symbol, new state, action (write symbol or move **L**eft one cell or move **R**ight one cell)

$$M = (\{p, q\}, \{x,y\}, \{x,y,\Diamond\}, \delta, p, \{q\})$$

Example

$M = (\{p, q\}, \{x,y\}, \{x,y,\Diamond\}, \delta, p, \{q\})$

Current	Read	Action	Next
p	$\Diamond$	R	p
p	x	x	q
p	y	y	q

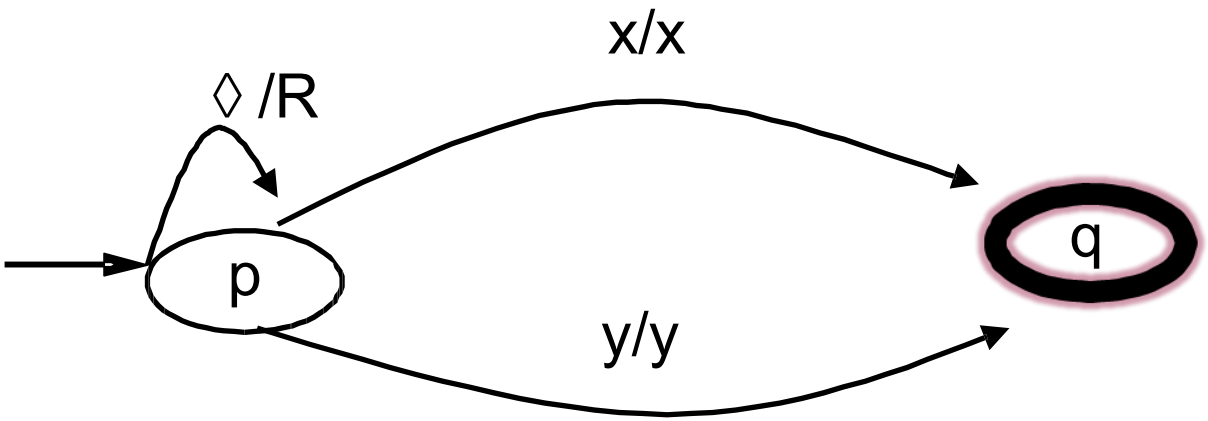
$\delta(p, \Diamond) = (p, R)$

$\delta(p, x) = (q, x)$

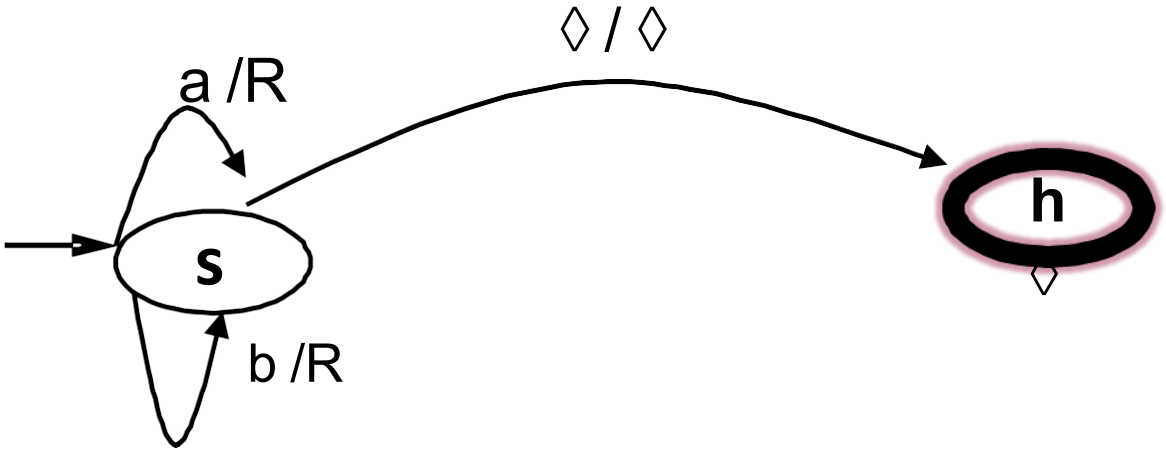
$\delta(p, y) = (q, y)$

Transition Diagrams

Current	Read	Action	Next
p	x	x	q
p	y	y	q
p	◊	R	p



Another Transition Diagram



- What does this simple Turing machine do?

It accepts finite strings of any length, including zero length, that comprise only the characters a or b i.e. (a|b)*
Why? Because it will crash (transition undefined) if any character other than a or b is on the tape